

RAG SİSTEMİ ÇALIŞMA RAPORU

Semantik Yapay Zeka Arama Altyapısı ve Kod Örnekleri

Rapor Künyesi ve Genel Bakış

- Konu: Retrieval-Augmented Generation (RAG) Teknolojisi
- Altyapı: ChromaDB, Sentence Transformers (all-MiniLM-L6-v2), Flask API
- Kaynak: 'RAG Explained For Beginners' Video Eğitimi (KodeKloud)
- Geliştirme Tarihi: 13 Ağustos 2026
- Hazırlayan: Gemini Notebook - Bilgi Mühendisliği Asistanı

Özet:

Bu rapor, kurumsal verilerin yapay zeka asistanlarına entegrasyonunda kullanılan RAG mimarisini ve KodeKloud laboratuvar ortamındaki çalışan uçtan uca kod örneklerini içermektedir.

1. GİRİŞ VE GELENEKSEL ARAMA PROBLEMLERİ

Bir kurumun sunucusunda 500 GB boyutunda çok büyük bir belge havuzu (çalışan el kitapları, ürün özellikleri, toplantı notları, SSS belgeleri vb.) bulunduğunda, bu verileri ChatGPT gibi bir yapay zeka asistanına bağlamak yaygın bir iş ihtiyacıdır. Ancak, bu süreçte geleneksel yöntemler kullanıldığında iki büyük engelle karşılaşılır:

- **Dosya Sınırları:** Klasik sohbet uygulamaları ve standart yapay zeka arayüzleri aynı anda yalnızca bir düzine dosyayı kabul edebilir. Bu nedenle 500 GB'lık bir kütüphaneyi doğrudan sisteme yüklemek imkansızdır.
- **Arama Verimsizliği ve Özetleme Kayıpları:** Belgelerin başlıklarında veya içeriklerinde anahtar kelimelere dayalı geleneksel arama algoritmaları geliştirmek, kullanıcı her soru sorduğunda tüm 500 GB'lık veri setini baştan sona taramayı gerektirir. Bu, aşırı derecede yavaş ve maliyetli bir yöntemdir. Belgeleri önceden özetleyerek daha küçük aranabilir parçalara dönüştürmek bir alternatif gibi görünse de, bu işlem önemli ayrıntıların kaybolmasına yol açar ve hassas yanıtlar üretme olasılığını ciddi şekilde düşürür.

2. ÇÖZÜM: RAG (RETRIEVAL-AUGMENTED GENERATION) NEDİR?

Bu sorunların üstesinden gelmek için iki yenilikçi yaklaşım birleştirilir: Kelime Gömme (Word Embedding) ve Vektör Veri Tabanları. Bu mimariye **RAG (Retrieval-Augmented Generation / Bilgi Geri Getirimi Artırılmış Üretim)** adı verilir.

- **Kelime Gömme (Word Embedding):** Bilgisayarlar kelimelerle değil, sayılarla işlem yapar. Kelime gömme teknolojisi, insan dilindeki kelimeleri ve cümleleri anlamsal ilişkilerini koruyarak sayısal vektörlere dönüştürür.
- **Vektör Veri Tabanı ve Parçalama (Chunking):** Belgeler, anlamsal anlamları korunarak vektör gömmeleri halinde bu özel veri tabanında saklanır. Belgeleri doğrudan saklamak yerine, onları küçük parçalara (chunks) ayırıp vektör veri tabanına kaydetmek, yapay zeka asistanının sınırlı bağlam penceresine (context window) sığacak verileri hızlıca geri getirmeyi sağlar.

RAG'ın Çalışma Mantığı

Günlük Hayattan Örnek: Bir şirket çalışanının yapay zeka asistanına 'Geçen yılki CodeCloud hizmet sözleşmesi hakkında bilgi verir misiniz?' diye sorduğunu varsayalım. Geleneksel bir sistem 'CodeCloud', 'hizmet', 'sözleşme' gibi statik kelimeleri ararken, RAG sistemi sorunun anlamsal içeriğini analiz eder. 'Geçen yıl' ifadesinin takvimsel karşılığını ve 'hizmet sözleşmesi' teriminin hukuki içeriğini anlayarak doğrudan ilgili belgelere ulaşır.

3. RAG YAPISININ ÜÇ TEMEL ADIMI

RAG mimarisi, bilgi akışını üç aşamada yönetir: **Bilgi Getirme (Retrieval)**, **Zenginleştirme (Augmentation)** ve **Yanıt Üretme (Generation)**.

1. Bilgi Getirme (Retrieval): Kullanıcının sorduğu soru (örneğin: 'Geçen yılki CodeCloud hizmet sözleşmesi nedir?'), tıpkı belgelerde yapıldığı gibi anında bir vektör gömmeye dönüştürülür. Elde edilen bu soru vektörü, vektör veri tabanında bulunan belge parçalarının vektörleriyle karşılaştırılır. Bu işleme **Semantik Arama (Semantic Search)** denir. Statik anahtar kelime eşleşmeleri yerine kelimelerin anlamsal yakınlığı ölçülür.

2. Zenginleştirme (Augmentation): Yapay zeka asistanları normalde sadece eğitim aşamasında edindikleri statik bilgilerle yanıt verirler. RAG sisteminde ise, semantik arama sonucunda elde edilen en alakalı güncel belge parçaları, çalışma anında (runtime) kullanıcının orijinal sorusuyla birleştirilerek isteme (prompt) enjekte edilir. Böylece modelde ince ayar (fine-tuning) yapmaya gerek kalmadan en güncel şirket verileriyle istem zenginleştirilir.

3. Yanıt Üretme (Generation): Son adımda yapay zeka asistanı, kendisine sunulan bu zenginleştirilmiş bağlamı (context) ve orijinal soruyu kullanarak yanıt üretir. Sistem, sağlanan güncel belgeleri kendi muhakeme yeteneğiyle (reasoning) işleyerek kullanıcıya tamamen doğru, güncel ve kaynaklandırılmış bir yanıt sunar.

4. VERİ KALİBRASYONU VE PARÇALAMA STRATEJİLERİ

Bir RAG sisteminin doğruluğu ve başarısı, verilerin sisteme nasıl beslendiğine ve nasıl kalibre edildiğine doğrudan bağlıdır. Bu kalibrasyon sürecinde üç kritik strateji bulunur: **Parçalama Stratejisi** (boyut ve örtüşme ayarı), **Embedding Stratejisi** (gömme modelinin seçimi) ve **Retrieval Stratejisi** (benzerlik eşiği ve ek filtrelerin ayarlanması).

Belge Türü	Parçalama Boyutu (Chunk Size)	Örtüşme (Overlap)	Stratejik Gerekçe
Hukuki Belgeler	Büyük (Örn: 500 - 800 karakter)	Düşük (Örn: 100 karakter)	Uzun, yapılandırılmış paragrafları ve sözleşme maddelerini bütün halinde korumak.
Müşteri Destek Sohbetleri	Küçük (Örn: 100 - 300 karakter)	Yüksek (Örn: 150 karakter)	Karşılıklı konuşma bağlamını ve soru-cevap bütünlüğünü kaçırmamak.

Güvenlik ve Doğruluk

Önemli Kalibrasyon Detayı: Kalibrasyon aşamasında, düşük kaliteli veya alakasız eşleşmeleri dışarıda tutmak amacıyla **benzerlik eşiği (similarity threshold)** kullanılır. Bu sınırın doğru belirlenmesi, sistemin halüsinasyon (uydurma yanıt) üretme ihtimalini minimize eder.

5. UYGULAMALI LABORATUVAR VE KODLAMA ADIMLARI

Aşağıdaki kod örnekleri, KodeKloud eğitim laboratuvarında uygulanan uçtan uca çalışan bir RAG sisteminin teknik geliştirme aşamalarını göstermektedir. Bu altyapıda yerel vektör veri tabanı olarak **ChromaDB**, gömme modeli olarak **Sentence Transformers (all-MiniLM-L6-v2)** ve web API katmanı için **Flask** kullanılmıştır.

A. Çevresel Geliştirme Ortamı Kurulumu

İlk olarak, yalıtılmış bir geliştirme ortamı kurulur ve gerekli paketler hızlı paket yöneticisi **uv** yardımıyla yüklenir:

```
# 1. Sanal ortam oluşturma ve etkinleştirme
python3 -m venv venv
source venv/bin/activate

# 2. Gerekli kütüphanelerin uv ile hızlı kurulumu
pip install uv
uv pip install chromadb sentence-transformers openai flask
```

B. Vektör Veri Tabanı İklendirme (Initialization)

ChromaDB veri tabanı yerel diskte kalıcı (persistent client) olarak çalıştırılır ve şirket belgelerinin vektörlerini barındıracak **tech_corp_docs** adında bir koleksiyon oluşturulur:

```
import chromadb

# Yerel diskte verileri saklayan kalıcı istemci oluşturma
client = chromadb.PersistentClient(path="./chroma_db")

# 'tech_corp_docs' koleksiyonunun iklendirilmesi
collection = client.get_or_create_collection(name="tech_corp_docs")
print("ChromaDB başarıyla iklendirildi.")
```

C. Metin Parçalama (Chunking Script)

Metinlerin anlamsal sınırlarını korumak için, belgeler 500 karakterlik bloklar halinde ve bağlam kayıplarını önlemek için 100 karakterlik bir örtüşme (overlap) payı ile bölünür:

```
def chunk_text(text, chunk_size=500, overlap=100):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        # Bir sonraki adıma kaydır (Örtüşmeyi koru)
        start += (chunk_size - overlap)
    return chunks

text_sample = "TechCorp çalışan el kitabı içeriği..."
chunks = chunk_text(text_sample, 500, 100)
print(f"Toplam parça sayısı: {len(chunks)}")
```

D. Kelime Gömme ve Benzerlik Analizi (Embeddings)

Kelimelerin ve cümlelerin anlamsal vektörlerini oluşturmak için kompakt ve yüksek performanslı **all-MiniLM-L6-v2** Sentence Transformer modeli kullanılır. Benzerliklerin ölçümü anlamsal aramayı simüle eder:

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Semantik gömme modelinin yüklenmesi
model = SentenceTransformer('all-MiniLM-L6-v2')

# Cümlelerin vektör temsillerinin çıkarılması
vec1 = model.encode("dogs allowed")
vec2 = model.encode("pets permitted")
vec3 = model.encode("remote work")

# Kosinüs Benzerliği (Cosine Similarity) fonksiyonu
cos_sim = lambda a, b: np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Benzer Cümleler (dogs vs pets):", cos_sim(vec1, vec2)) # Yüksek skor
print("Farklı Cümleler (dogs vs remote):", cos_sim(vec1, vec3)) # Düşük skor
```

E. Veri Yükleme Hattı (Data Ingestion)

Sistem elindeki ham belgeleri (TechCorp el kitapları, toplantı notları vb.) okur, parçalara böler, gömme vektörlerini çıkarır ve ilişkili meta verilerle birlikte ChromaDB'ye ekler:

```
import os

vault_path = "./tech_corp_vault"
for filename in os.listdir(vault_path):
    with open(os.path.join(vault_path, filename), "r") as f:
        content = f.read()

    # Boyut 500, Adım (Stride) 400 olacak şekilde parçalama
    doc_chunks = chunk_text(content, chunk_size=500, overlap=100)

    for idx, chunk in enumerate(doc_chunks):
        # Gömme vektörünü numpy dizisinden listeye çeviriyoruz
        embedding = model.encode(chunk).tolist()

        # Vektör veri tabanına döküman, vektör ve meta-verilerin eklenmesi
        collection.add(
            documents=[chunk],
            embeddings=[embedding],
            metadatas=[{"source": filename, "chunk": idx}],
            ids=[f"{filename}_{idx}"]
        )

print("Tüm belgeler başarıyla veri tabanına işlendi.")
```

F. Web Entegrasyonu (Flask API) ve RAG Akışı

Uygulamanın son aşamasında, semantik aramayı dış dünyaya açan ve kullanıcılardan gelen sorulara bağlı olarak RAG akışını (Retrieve, Augment, Generate) tetikleyen bir Flask uygulaması port 5000 üzerinde ayağa kaldırılır:

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/ask", methods=["POST"])
def ask_question():
    user_query = request.json.get("query")
    query_vector = model.encode(user_query).tolist()

    # Adım 1: Getirme (Retrieval) - Alakalı 2 belge parçasını al
    search_results = collection.query(
        query_embeddings=[query_vector],
        n_results=2
    )

    # Adım 2: Zenginleştirme (Augmentation)
    retrieved_docs = search_results['documents'][0]
    prompt_context = "\n".join(retrieved_docs)

    # Adım 3: Üretim (Generation) - Prompt LLM'e gönderilir (Temsili)
    augmented_prompt = f"Başlam: {prompt_context}\n\nSoru: {user_query}"

    return jsonify({
        "status": "success",
        "retrieved_context": retrieved_docs,
        "augmented_prompt": augmented_prompt
    })

if __name__ == "__main__":
    app.run(port=5000)
```